Université Internationale de Rabat

AI & Big Data Program — S8 Integrated Project

Academic Year 2025–2026

Smart Document Analyst

A Multi-Agent AI System for Document Classification, Extraction, Summarization, and Report Generation

Realized by:

Benmouma Salma
Gassi Oumaima

Supervised by:

Prof. Hakim Hafidi

April 2026

Abstract

This report presents the design, implementation, and evaluation of a multi-agent AI system for automated document analysis. The system employs three specialist agents orchestrated by CrewAI to classify documents using a CNN deep learning model, extract and summarize text content, and generate structured analysis reports. A fine-tuned ResNet-18 model trained on the RVL-CDIP dataset achieves 85.62% test accuracy across 8 document classes. The system includes a human-in-the-loop checkpoint, comprehensive error handling, and JSON-based action logging. This report details the architectural decisions, model training, agent design, and evaluation results.

Contents

1 Introduction

1.1 Context and Motivation

In modern organizations, the volume of documents—invoices, letters, reports, forms—continues to grow rapidly. Manually classifying, reading, and summarizing these documents is time-consuming and error-prone. An intelligent system capable of automating this pipeline would significantly improve productivity and accuracy.

1.2 Project Objectives

The objective of this project is to design and build a **multi-agent AI system** where specialized agents collaborate to:

1. **Classify** documents into predefined categories using a CNN deep learning model.
2. **Extract** text from documents (native PDF or OCR for scanned documents).
3. **Summarize** the extracted content using a large language model (Gemini API).
4. **Generate** a structured professional report combining all analysis results.

1.3 Technical Constraints

As specified in the project brief, the system must satisfy the following requirements:

- At least 2 specialist agents + 1 orchestrator
- A PyTorch deep learning model trained by the team, integrated as a functional tool
- At least 2 tools with clear input/output schemas
- A human-in-the-loop checkpoint requiring human approval
- Error handling for tool failures, invalid inputs, and unexpected outputs
- JSON logging with timestamps for every agent action

2 System Architecture

2.1 High-Level Overview

The system follows a **sequential pipeline** architecture where documents flow through four stages: classification, human approval, extraction & summarization, and report generation. Each stage is handled by a specialist agent equipped with dedicated tools.

```
User Input (PDF/Image)
      |
[Orchestrator Agent] --- Manages workflow
      |
[Classifier Agent] --- CNN Model Tool (PyTorch)
      |
[HITL Checkpoint] --- Human approves/corrects classification
      |
[Extractor & Summarizer Agent] --- OCR + LLM Tools
      |
[Report Generator Agent] --- Report Builder Tool
      |
Final Analysis Report (Markdown/PDF)
```

Figure 1: System pipeline architecture

2.2 Agent Roles and Responsibilities

Table 1: Agent definitions and their tools

Agent	Tools	Responsibility
Classifier	`cnn_classify_tool`	Classify document type using the trained CNN model
Extractor & Summarizer	`ocr_extract_tool`, `llm_summarize_tool`	Extract text via OCR and generate a type-aware summary
Report Generator	`report_builder_tool`	Produce a structured Markdown/PDF report

2.3 Orchestration with CrewAI

The agents are orchestrated using **CrewAI** with a sequential process. The orchestrator:

- Routes tasks to agents in the correct order
- Passes context (classification results) between agents
- Enforces the HITL checkpoint between classification and extraction
- Aggregates the final output

The system supports two execution modes:

- **Standard mode**: Fully automated sequential pipeline via `crew.run()`
- **HITL mode**: Manual orchestration with an interactive classification approval checkpoint via `crew.run_with_hitl()`

2.4 Project Structure

Listing 1: Repository structure

```
smart-document-analyst/
        config/
              agents.yaml          # Agent role definitions
              tasks.yaml           # Task pipeline definitions
        src/
              main.py              # CLI entry point
              crew.py              # CrewAI crew orchestration
              agents/              # Agent definitions
              tools/               # Tool implementations
              models/              # PyTorch model definition
              utils/               # Logger, HITL, preprocessing
        model/
              document_classifier.pt  # Trained model weights
        notebooks/
              train_on_colab.ipynb    # Training notebook
        tests/                       # Unit tests
```

3 Deep Learning Model

3.1 Dataset: RVL-CDIP

We use the **RVL-CDIP** (Ryerson Vision Lab – Complex Document Information Processing) dataset, a widely-used benchmark for document image classification. From the full 16-class dataset, we selected a **subset of 8 classes** to maintain feasibility on Google Colab's free tier:

Table 2: Selected document classes (2,000 images per class)

Class	Description
letter	Formal correspondence
form	Structured forms with fields
invoice	Financial billing documents
resume	Curriculum vitae / CVs
scientific_publication	Academic papers
email	Electronic mail printouts
memo	Internal memoranda
advertisement	Marketing materials

Total dataset size: **16,000 images**, split as follows:

- Training: 12,800 images (80%)
- Validation: 1,600 images (10%)
- Test: 1,600 images (10%)

3.2 Model Architecture

We use a **fine-tuned ResNet-18** pretrained on ImageNet. The architecture choices are:

- **Backbone**: ResNet-18 with ImageNet pretrained weights
- **Frozen layers**: `conv1`, `bn1`, `layer1`, `layer2` (early feature extraction layers)
- **Trainable layers**: `layer3`, `layer4`, and the custom classification head
- **Classification head**: Dropout(0.3) → Linear(512, 256) → ReLU → BatchNorm → Dropout(0.15) → Linear(256, 8)

Rationale: Fine-tuning a pretrained model leverages learned visual features while adapting to document-specific patterns. Freezing early layers reduces training time and prevents overfitting on our relatively small dataset.

Parameters: 11,310,408 total; trainable parameters focus on the deeper layers and classification head.

3.3 Training Configuration

Table 3: Training hyperparameters

Parameter	Value
Input size	224×224 (grayscale → 3-channel)
Batch size	32
Optimizer	AdamW (lr=10^{-4}, weight_decay=10^{-4})
Scheduler	CosineAnnealingLR ($T_{max} = 20$, $\eta_{min} = 10^{-6}$)
Loss function	CrossEntropyLoss
Epochs	20 (early stopping, patience=5)
Device	NVIDIA T4 GPU (Google Colab)

Data augmentation (training only): RandomCrop(224), RandomRotation(5°), RandomAffine (translate=5%, scale=95–105%), ColorJitter (brightness=0.2, contrast=0.2).

3.4 Training Results

Training converged in **18 epochs** before early stopping was triggered. The best validation accuracy was achieved at epoch 13.

Table 4: Training progression (selected epochs)

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	0.9478	67.51%	0.7341	74.62%
5	0.3796	87.30%	0.5043	83.81%
10	0.1881	93.80%	0.5024	86.19%
13	0.1231	95.78%	0.5075	**87.62%**
18	0.0693	98.02%	0.5397	87.62%

3.5 Test Set Evaluation

Table 5: Per-class classification results on the test set

Class	Precision	Recall	F1-Score	Support
letter	0.88	0.95	0.92	200
form	0.91	0.95	0.93	191
invoice	0.78	0.72	0.75	207
resume	0.82	0.83	0.83	204
scientific_publication	0.83	0.71	0.76	201
email	0.81	0.84	0.82	206
memo	0.89	0.90	0.90	196
advertisement	0.92	0.94	0.93	195
Overall Accuracy		**85.62%**		
Macro Avg	0.86	0.86	0.86	1600

Analysis: The model performs best on advertisement (F1=0.93) and form (F1=0.93), and weakest on invoice (F1=0.75) and scientific_publication (F1=0.76). The confusion between invoice and form is expected given their visual similarity (both contain structured fields and tables).

4 Tools Implementation

Each tool is implemented as a CrewAI `BaseTool` subclass with a Pydantic input schema.

4.1 Tool 1: CNN Classification Tool

- **Input**: File path to a document (PDF, PNG, JPG)
- **Output**: JSON with predicted class, confidence score, and top-3 predictions
- **Process**: Loads the trained PyTorch model, preprocesses the document image (PDF pages are rendered via PyMuPDF), runs inference with `torch.no_grad()`, and applies softmax for probabilities

4.2 Tool 2: OCR Extraction Tool

- **Input**: File path to a document
- **Output**: JSON with extracted text, page count, and extraction method
- **Strategy**: Tries PyMuPDF native text extraction first; if the result is too short (<50 characters, indicating a scanned document), falls back to Tesseract OCR

4.3 Tool 3: LLM Summarization Tool

- **Input**: Extracted text, document type, maximum summary length
- **Output**: JSON with summary, key points, and word count
- **Features**: Adapts the prompt based on document type (e.g., invoices focus on amounts; resumes focus on skills). Includes retry logic with exponential backoff and an extractive fallback when the API is unavailable

4.4 Tool 4: Report Builder Tool

- **Input**: Classification, extraction, and summary results as JSON
- **Output**: JSON with the path to the generated report file
- **Format**: Generates a professional Markdown report with optional PDF conversion via `fpdf2`

5 Human-in-the-Loop

The HITL checkpoint is triggered after classification and before downstream processing. It presents the CNN's prediction to the user and offers three options:

1. **Approve**: Accept the classification and proceed
2. **Override**: Correct the class label (the corrected label propagates to the summarizer and report)
3. **Reject**: Stop the pipeline entirely for this document

This is not decorative—the user's decision has genuine functional impact on the system's behavior. If the user overrides the class, the Extractor Agent adapts its summarization style to the corrected document type, producing a more relevant summary.

6 Error Handling and Logging

6.1 Error Handling

Every tool implements defensive error handling:

Table 6: Error handling scenarios

Scenario	Handling
File not found	Returns descriptive error JSON, no crash
Unsupported format	Validates extension, returns supported formats list
CNN model missing	Graceful error with download instructions
OCR returns empty	Falls back to Tesseract; if still empty, warns user
Gemini API failure	3 retries with exponential backoff, then extractive fallback
Invalid LLM output	Parse validation and re-prompting

6.2 Logging

All agent actions are logged to `logs/agent_actions.jsonl` in structured JSON format:

Listing 2: Example log entry

```
{
  "timestamp": "2026-04-26T14:23:01Z",
  "agent": "ClassifierAgent",
  "action": "cnn_classify_tool",
  "input": {"file_path": "data/sample_docs/invoice.pdf"},
  "output": {"class": "invoice", "confidence": 0.943},
  "status": "success",
  "duration_ms": 1230
}
```

7 Evaluation and Robustness

7.1 CNN Model Evaluation

The CNN model was rigorously evaluated with:

- **Accuracy**: 85.62% on a held-out test set of 1,600 images
- **Confusion matrix**: Analyzed to identify class-pair confusions
- **Per-class metrics**: Precision, recall, and F1-score for all 8 classes
- **Training curves**: Monitored for overfitting (early stopping triggered at epoch 18)

7.2 Honest Failure Analysis

- **Invoice vs. Form confusion**: Both document types share visual features (tables, structured fields), leading to the lowest F1 scores

- **Scientific publication recall**: At 71%, some academic papers are misclassified as letters or memos, likely due to text-heavy layouts that resemble correspondence
- **Overfitting gap**: Training accuracy (98%) vs. test accuracy (85.6%) indicates some overfitting despite data augmentation and dropout. More training data or stronger regularization could help

8 Challenges and Lessons Learned

1. **Dependency conflicts on Colab**: NumPy 2.0 broke compatibility with scikit-learn and scipy. Solution: pinning `numpy<2.0.0` and restarting the runtime.
2. **HuggingFace dataset loading**: The `rvl_cdip` shorthand was deprecated. Solution: using the full path `aharley/rvl_cdip`.
3. **GitHub notebook rendering**: Jupyter widget metadata from tqdm caused "Invalid Notebook" errors. Solution: stripping `metadata.widgets` before committing.
4. **Agent design**: The key insight is that each agent must have a *distinct, justified role*. Merging tools into fewer agents would reduce the system's modularity and make it harder to debug.

9 Conclusion

We have designed, built, and evaluated a multi-agent AI system for document analysis that satisfies all project requirements. The system demonstrates genuine collaboration between specialized agents, each with a distinct role and dedicated tools. The CNN deep learning model, trained on real document data, serves as a functional tool—not decoration—within the agent pipeline.

The system achieves 85.62% classification accuracy, includes a meaningful human-in-the-loop checkpoint, and handles errors gracefully without crashing. Every agent action is logged with timestamps in structured JSON format.

Future work could include:

- Training on the full 16-class RVL-CDIP dataset for broader coverage
- Adding a Streamlit/Gradio web interface for non-technical users
- Migrating to LangGraph for more complex agent communication patterns
- Fine-tuning a smaller language model locally to remove API dependency

References

1. Harley, A.W., Ufkes, A., Derpanis, K.G. (2015). *Evaluation of Deep Convolutional Nets for Document Image Classification and Retrieval.* ICDAR.
2. He, K. et al. (2016). *Deep Residual Learning for Image Recognition.* CVPR.
3. CrewAI Documentation. `https://docs.crewai.com/`

4. PyTorch Documentation. https://pytorch.org/docs/

5. Google Gemini API. https://ai.google.dev/

6. RVL-CDIP Dataset. https://huggingface.co/datasets/aharley/rvl_cdip